

The engineered architecture for **HiveIDE v6.0** represents a shift from reactive text generation to a **proactive execution ecosystem** 1. It is a **local-first, multi-agent "digital organism"** designed to convert natural language intent into shippable code in under 5 minutes while maintaining a \$0 default cost 2-4.

1. Infrastructure Architecture

The system is distributed as a **single Go 1.22 static binary** (≈28 MB) that embeds its own libraries (Ollama, SQLite, tree-sitter), ensuring zero-dependency portability across Linux, macOS, and Windows 5-7.

System Infrastructure Graph (C4 Container Model)

```
C4Container
title HiveIDE v6.0 Infrastructure - Sovereign Kernel
Person(dev, "Developer", "Natural Language Intent")

Container_Boundary(hive, "HiveIDE Ecosystem") {
    Container(web, "Unified UI", "Next.js/Monaco", "Symbolect-driven
interface & real-time monitoring")
    Container(kernel, "Merlin_Ω Kernel", "Go 1.22", "Orchestrates
swarm topology via LangGraph")
    Container(runtime, "Agent Sandbox", "Docker + gVisor", "Isolated,
ephemeral execution for tests/patching")

    ContainerDb(kg, "Knowledge Graph", "Neo4j", "AST relationships,
call graphs, and file dependencies")
    ContainerDb(vector, "Code Memory", "Qdrant", "Semantic indexing
of symbols and documentation")
    ContainerDb(state, "Evolving Memory", "Redis/SQLite", "Session
state, task logs, and UKG anchors")
}

System_Ext(llm_local, "Local Model", "DeepSeek-Coder 7B GGUF",
"Primary implementation & logic")
System_Ext(llm_burst, "Cloud Burst", "Gemini 2.5 Pro / Claude 3.5",
"Heavy planning & architectural reasoning")
System_Ext(git, "VCS", "GitHub/GitLab", "PR creation and branch
management")

Rel(dev, web, "Plain English prompts", "WebSocket")
Rel(web, kernel, "Compressed intent", "gRPC")
Rel(kernel, runtime, "Dispatches commands", "bwrap/Docker API")
Rel(kernel, kg, "Graph RAG queries", "Bolt")
Rel(kernel, llm_local, "Inference", "Ollama API")
Rel(kernel, llm_burst, "Burst planning (>70% risk)", "HTTPS/API")
```

2. Governing Structures & Additional Protocols

A. The Multi-Knight Orchestration (Map-Reduce Protocol)

HiveIDE utilizes a **Map-Reduce Swarm topology** 8, 9. Instead of one agent handling the entire task, the **Merlin_Ω Kernel** decomposes intent into a **Directed Acyclic Graph (DAG)** of sub-tasks 8, 10, 11.

- **PHASE A (MAP):** Specialized agents (Knights) like **Oracle** (Planning), **Forge/Lucas** (Building), and **Sentinel** (Security) process atomic steps in parallel 8, 11.
- **PHASE B (REDUCE):** Results are consolidated by **Mason** (Bundling) and **Glyph** (Memory Compression) into a single shippable artifact 12.

B. The Titanium Laws (Governance)

1. **The 10-Line Iron Gate:** Any code diff exceeding 10 net lines triggers a mandatory **Human-in-the-Loop (HITL)** approval gate 8, 13, 14.
2. **Kinetic Purity:** All AI-generated code execution must occur within **ephemeral, rootless containers** (gVisor/bwrap) with no default network access 8, 15, 16.
3. **Zero-Trust Security:** Every patch is scanned for secrets and vulnerabilities by **Sentinel** using adversarial simulation before application 15, 17, 18.
4. **REVERT-BASE:** The system automatically reverts to the last stable git-SHA if tests fail or an unauthorized security breach is detected 6, 18, 19.

C. Symbolect v2.0 (Compression Protocol)

To maximize efficiency on 8GB RAM hardware, HiveIDE converts verbose intent into **high-density visual glyphs** 17, 20, 21.

- **Example:** "Add dark-mode toggle with localStorage" → [🌙↔️🔗📦] 17, 22, 23.
- **Efficiency:** Slashes prompt tokens by **90-96%**, allowing 15+ agents to operate within a single 4k context window 17, 22, 24.

3. Command Prompts & Triggers

The system uses a **40-token NLP shim** to map plain English to 3-letter triggers or specialized slash commands 25-28.

Command Trigger,Persona Summoned,Primary Task

hive pln,"Aurelio ""Oracle""","NL spec → 3-step Task DAG 29, 30"

hive forge,"Kai ""Forge"" (Lucas)","End-to-end stack build & Dockerization 30, 31"

hive dbg,"Marta ""Debug""","Traceback analysis & minimal AST patching 29, 32"

hive scan,"Ivan ""Sentinel""","Zero-trust linting & secret scanning 32, 33"

hive swarm,Full Crew,"Parallel ""race"" to generate/vote on best fix 28, 30, 34"

//BUNDLE,Mason,"Fusing modules into single shippable artifacts 12, 35"

4. User Comprehensive Guide

Initial Setup (30 Seconds)

1. **Install:** Execute `curl -L https://github.com/hive-agentic/hive/releases/latest/download/hive-linux-amd64 | sudo tee /usr/local/bin/hive && sudo chmod +x /usr/local/bin/hive` 5, 36.
2. **Initialize:** Run `hive init --demo` to scaffold a project and index code into the **Neo4j/Qdrant memory layer** 36, 37.

The Development Loop (No-Code Flow)

- **Intent Verbalization:** Describe your feature in plain English: `hive "add a /health endpoint that returns JSON"` 38, 39.
- **Interactive Clarification:** If the prompt is ambiguous, the **Clarifier Agent** will pop up a chat: *"Should users log in with Google or Email?"* 40, 41.
- **Planning Mode:** Review the **Implementation Plan artifact** (Implementation Plan, Goal, Tasks). You can **annotate** specific lines to give feedback 42, 43.
- **Automatic Execution (PIV Loop):** The agents **Plan, Implement, and Validate** 44, 45.
- The **Browser Agent** launches a sandboxed browser to interact with the UI, recording screenshots as "proof of work" 46-48.
- **One-Click Review:** View the unified diff in the **Terminal User Interface (TUI)** 39, 49, 50.
- **Merge & PR:** Hit `y` to have the **GitOps Agent** commit, push, and open a Pull Request with attached test evidence 13, 51.

Customization Levers

- **Rules:** Create a `.hive-rules` file to enforce team coding standards (e.g., "max 8 lines per function," "require tests") 52, 53.
- **Cloud Burst:** Use the `--burst gemini` flag for complex refactors or multi-repo migrations that exceed local compute capacity 54, 55.
- **Language Packs:** Add new languages via a simple JSON config in the repo root; the system auto-detects languages via **file sniffing** 56-58.